

HisabDo AI Copilot

Implementation Documentation

Documented by: Laiba Yousuf

Department: 3

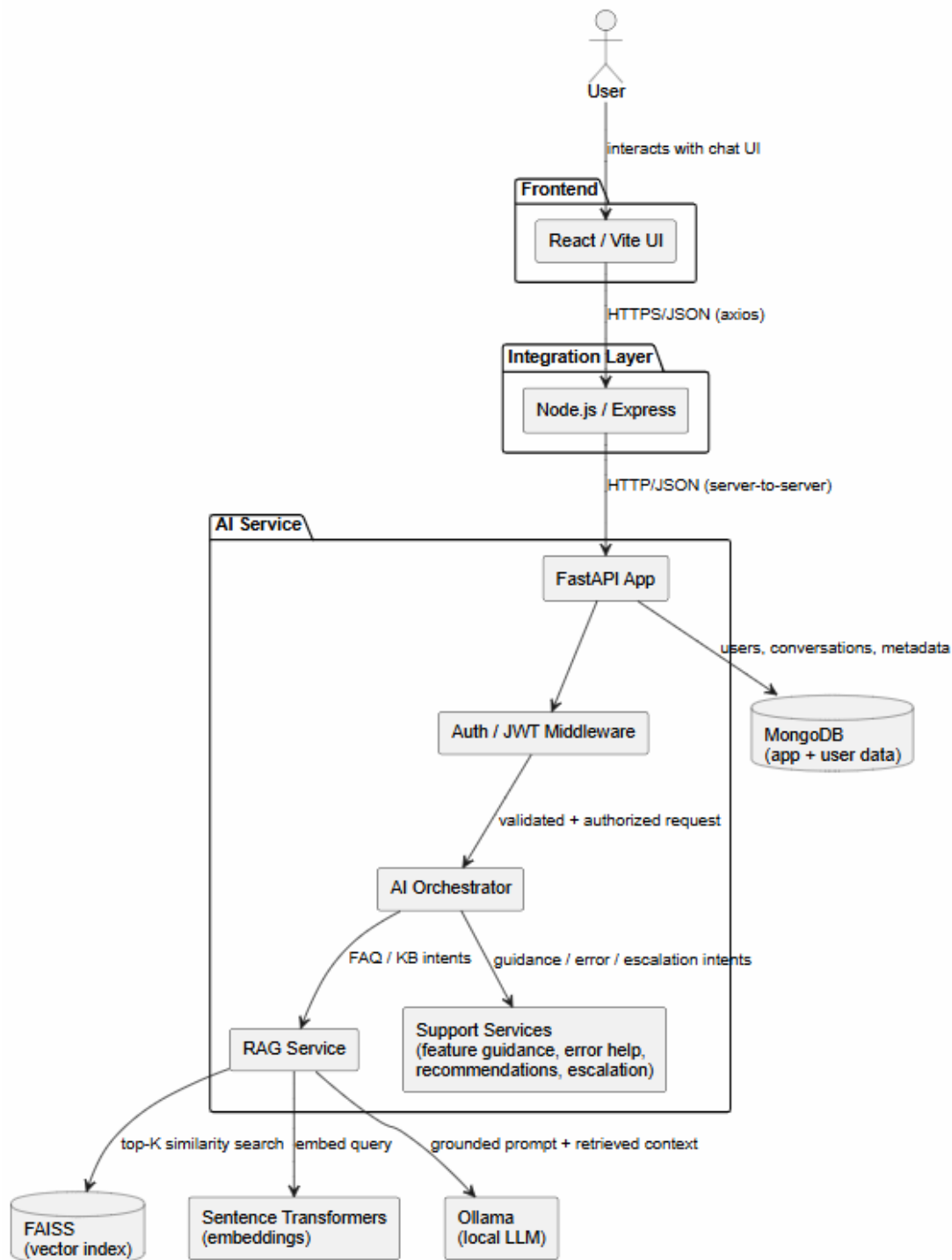
Track: AI/ML

This document consolidates the chatbot architecture, request/response workflow, API contracts, orchestrator logic, RAG/knowledge-base design, security rules, and testing results scattered across Document AI POC and Integration Plan/, Identify Technical Dependencies/, and HisabDo_RAG_Chatbot/ into one developer-ready reference.

Table of Content

1. Finalized Chatbot Architecture.....	3
2. Complete Request–Response Workflow.....	4
3. API Integration.....	5
Core endpoints	5
Request format	5
Success response	6
Error response	6
HTTP status codes	6
4. React → Express → FastAPI Communication	7
5. AI Orchestrator Routing	7
6. Intent Categories	8
7. FAQ / RAG Integration	8
8. Knowledge-Base Requirements	9
9. Financial-Data Access Rules	10
10. Authentication and Authorization.....	10
11. Security and Privacy Requirements	11
12. Anti-Hallucination Controls.....	12
13. Error Handling and Fallback Behavior	13
14. Testing Results.....	14
Functional / intent test cases	14
RAG retrieval evaluation (12-query labeled set)	14
15. Current POC Limitations	15
16. Developer Setup / Integration Checklist	15

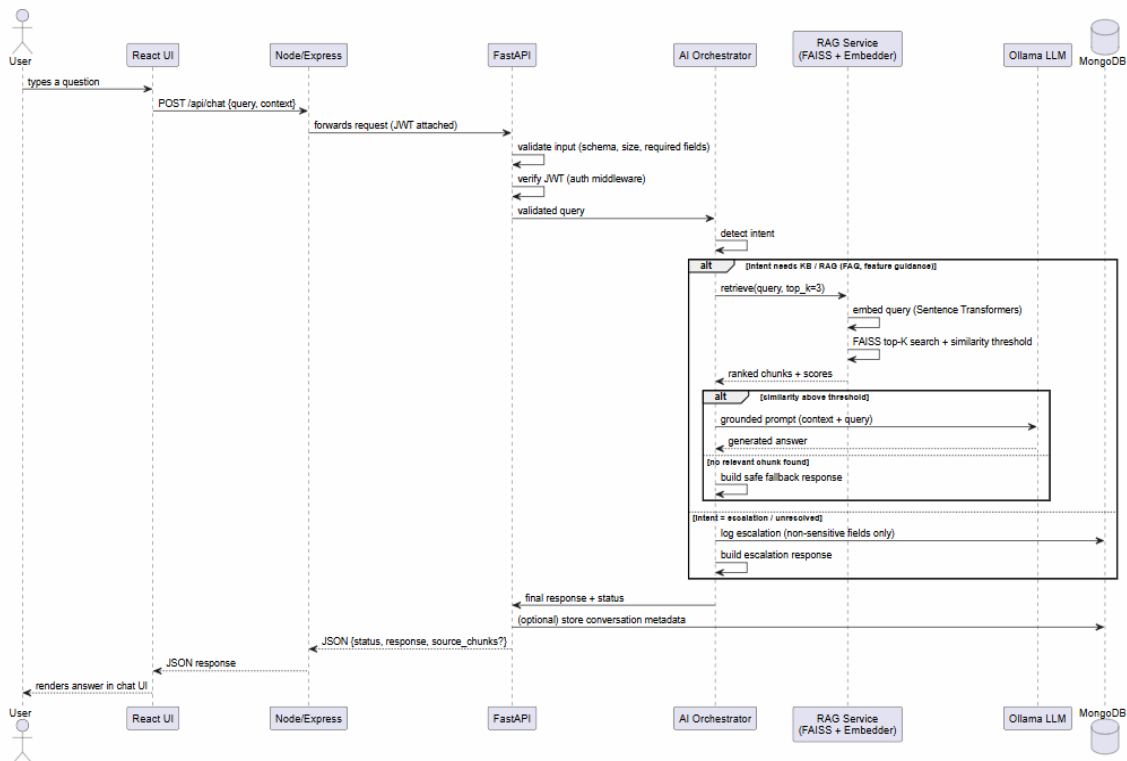
1. Finalized Chatbot Architecture



Layer responsibilities

Layer	Responsibility
React / Vite	Chat UI, auth screens, renders AI responses and (optionally) retrieved sources; never talks to MongoDB/FAISS/Ollama directly; never holds secrets.
Node/Express	Optional integration hop: forwards requests, applies CORS, normalizes errors, can centralize session handling. React may also call FastAPI directly for simple deployments — this hop is configurable, not mandatory.
FastAPI	Validates input, enforces auth, hosts the AI Orchestrator, exposes REST endpoints, talks to MongoDB/FAISS/Ollama.
MongoDB	Application data — users, conversation logs, KB metadata. Never exposed to the frontend directly.
FAISS	Local vector similarity search over the knowledge-base embeddings (Qdrant is the planned production replacement).
Ollama	Local LLM inference, called only with a grounded prompt built from retrieved KB chunks.

2. Complete Request–Response Workflow



Text form:

```
User → React → Express → FastAPI (validate + auth)

→ AI Orchestrator (intent detection)

→ [RAG retrieve (FAISS) → similarity check → Ollama generate] OR [safe fallback / escalation]

→ Response Validation & Safety Layer

→ JSON Response → Express → React → User
```

3. API Integration

All AI endpoints are served by FastAPI and consumed by React either directly or via the Express integration layer. POST is used for anything that sends data for processing; GET is used for status/health checks.

Core endpoints

```
GET /health

GET /health/database

GET /health/llm

GET /health/vector-store

POST /api/auth/login

POST /api/rag/query      # FAQ / knowledge-base questions

POST /ai/help/context    # context-aware assistance

POST /ai/personalized-recommendation
```

Request format

```
{

  "query": "How can I add a new expense?",

  "context": { "feature": "expenses" }
```

```
}
```

Success response

```
{  
  
  "status": "success",  
  
  "response": "Open the Expenses section and select Add Expense.",  
  
  "intent": "feature_guidance",  
  
  "source_chunks": [  
  
    { "section_title": "Adding an Expense", "similarity_score": 0.91 }  
  
  ]  
  
}
```

Error response

```
{ "status": "error", "message": "Request data is required." }
```

HTTP status codes

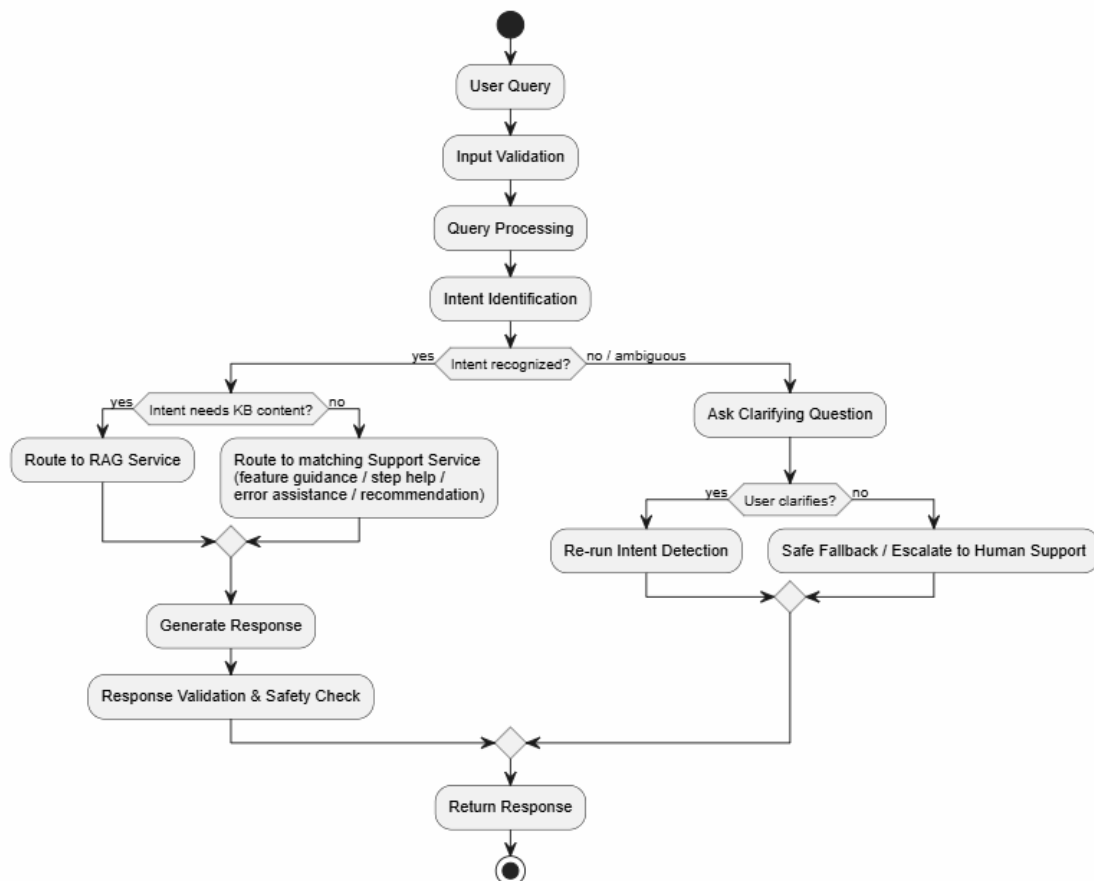
Code	Meaning
200	Request successful
400	Invalid request (missing/malformed input)
401	Missing or invalid authentication
403	Authenticated but not authorized for this resource
404	Resource not found
429	Rate limit exceeded
500	Internal server error (generic message only — see §11)

4. React → Express → FastAPI Communication

- **React → Express:** axios, HTTPS/JSON, carries the user's session/JWT in the Authorization header. No provider credentials (Mongo URI, Ollama URL, JWT secret) ever reach the browser.
- **Express → FastAPI:** server-to-server HTTP/JSON. Express applies CORS restriction (CORS_ORIGINS), centralizes error normalization (so React always receives the same {status, message} shape regardless of which internal service failed), and can inject correlation/request IDs for tracing.
- **Direct-call option:** for a simpler deployment, React may call FastAPI directly for endpoints that don't need the Express hop. This is a configuration choice (VITE_API_BASE_URL), not an architectural requirement — keep both paths supported so either topology works without a frontend rewrite.
- **FastAPI internal calls:** FastAPI is the only service that talks to MongoDB, FAISS, and Ollama. Nothing upstream of FastAPI is allowed to reach those directly.

5. AI Orchestrator Routing

The AI Orchestrator is the component inside FastAPI that decides **what the user actually wants** and routes the request to the correct service, instead of every endpoint trying to handle every kind of question.



Routing is intentionally modular — each intent maps to one service module, so a new capability can be added by adding a service and a routing rule, without touching the others.

6. Intent Categories

Intent	Purpose	Example query	Routed to
Feature Guidance	Explain how to use a feature	"How do I add an expense?"	feature_guidance
Step Help	Step-by-step instructions	"Show me the steps to manage my expenses."	step_help
Error Assistance	Troubleshoot a problem	"My expense is not being saved."	error_assistance
FAQ	Answer common questions	"What is HisabDo?"	RAG / faq
Help Article	Suggest relevant help content	—	help_articles
Context Help	Context-aware assistance	"I can't find the expense option."	context_support
Recommendation	Personalized recommendation from expense data	Structured {expenses:[...]} payload	personalized_recommendation
Support Automation	Handle routine support requests	—	support_automation
Escalation	Hand off to human support	"It's still not working." (after clarification)	escalation

Ambiguous intent (e.g. "It is not working.") is never guessed — the orchestrator asks a clarifying question first (see §12–13).

7. FAQ / RAG Integration

The RAG pipeline (built and evaluated in HisabDo_RAG_Chatbot/) is the retrieval engine behind the FAQ and feature-guidance intents.

Ingestion (offline, run once / on KB update)

Documents (.md) → cleaning → chunking (on ## boundaries) → embeddings → FAISS index + metadata

Runtime (per query)

User query → embed query (same model as ingestion) → FAISS top-K search

→ similarity threshold filter → grounded prompt → Ollama → response

Rule: the embedding model used for indexing must match the model used for querying. If it changes, the FAISS index must be rebuilt from scratch.

Grounding discipline: the LLM is only ever given the retrieved chunks as context — it is not asked to answer from general knowledge. If no chunk clears the similarity threshold, the system returns the safe-fallback response instead of calling the LLM with empty/weak context (see §12).

Measured retrieval quality (12-query hand-labeled evaluation set, reproduced twice from a clean environment):

Metric	Result
Hit@1	83.3% (10/12 queries retrieved the correct chunk as the top result)
Hit@3	100% (12/12 queries retrieved the correct chunk within the top 3)
Mean Reciprocal Rank	0.903

Known substitution: the current prototype uses TF-IDF vectorization instead of the architecture's specified Sentence Transformers, because the development sandbox blocked network access to huggingface.co. This is documented, not silent, and everything downstream (FAISS index, chunking, similarity search, evaluation) is fully real. Swapping in real Sentence Transformer embeddings before production is a tracked follow-up, not a redesign — embedder.py is already isolated behind the same interface.

8. Knowledge-Base Requirements

- **Source of truth:** only approved content — official FAQs, help articles, application documentation, verified feature descriptions. Unverified or user-supplied claims are never treated as authoritative KB content.
- **Current structure** (HisabDo_RAG_Chatbot/knowledge-base/):

knowledge-base/

└─ faqs/ (general, accounts, transactions, backup)

└─ features/	(khata, analytics, voice-entry, pdf-export, customers)
└─ guides/	(getting-started, expense-tracking, managing-customers)
└─ blogs/	(daily-khata, digital-ledger)

- **Chunking rule:** split on markdown `##` headers *before* stripping markdown syntax, so a question and its answer stay in the same chunk (a real bug caught during development — splitting after cleaning separated questions from their answers).
- **Update process:** adding/editing a .md file requires re-running the ingestion pipeline (`build_index()`) so FAISS reflects the change — there is no live/incremental indexing in the current POC.
- **Future improvement:** connect KB updates to user feedback and frequently-searched, currently-unanswered questions, so gaps get closed based on real usage.

9. Financial-Data Access Rules

- The AI service must only receive the **minimum** financial data required for the specific request (e.g. amount + category for a recommendation) — never a full transaction history or account balance unless the intent explicitly requires it.
- Financial/expense data submitted for recommendations is processed **in-memory for the request** and is not persisted unless the user's own application flow already stores it (e.g. their existing expense record in MongoDB) — the AI layer itself should not create a second, redundant copy of financial data.
- The frontend **never** queries MongoDB directly and never receives database credentials — all financial data access goes through authenticated FastAPI endpoints.
- Financial values must never be written to logs (see §11) — not in request logs, not in error traces, not in escalation records.
- Any future feature that lets the chatbot *read* a user's financial data (e.g. "how much did I spend on food this month?") must scope that read to the authenticated user's own `user_id` only, enforced server-side — never trust a client-supplied user identifier for financial lookups.
- Escalation records may reference *that* a financial question occurred, but should not embed raw amounts/account details in the escalation payload beyond what human support strictly needs to act on the ticket.

10. Authentication and Authorization

- **Mechanism:** JWT-based authentication (PyJWT, `passlib[bcrypt]` for password hashing).
- **Flow:** POST `/api/auth/login` issues an access token (`ACCESS_TOKEN_EXPIRE_MINUTES`, HS256). Protected endpoints require a valid Authorization: Bearer <token> header, verified by FastAPI middleware before the request reaches the Orchestrator.

- **Authorization:** authentication confirms *who* the user is; authorization confirms *what* they can access. Every request that touches user-specific data (recommendations, history-dependent features) must check that the token's `user_id` matches the data being requested — a valid token does not imply access to *any* user's data.
- **Secrets:** `JWT_SECRET` and all credentials live in environment variables (`.env`, never committed) — see `.env.example` for the required variable names.
- **Frontend rule:** React holds only the access token (in memory or a secure storage mechanism appropriate to the deployment) — it never holds the JWT secret, database credentials, or the Ollama/FAISS connection details.

11. Security and Privacy Requirements

- **Authentication & authorization** enforced on all protected endpoints (§10).
- **Input validation:** required fields, data types, format, size limits, and rejection of empty/unexpected payloads, on every incoming request.
- **Rate limiting:** required before production to prevent abuse, resource exhaustion, and runaway inference cost.
- **HTTPS:** required for all production traffic, frontend↔Express↔FastAPI.
- **Secrets management:** no API keys, JWT secrets, or DB credentials in source code — environment variables / secret storage only; `.env` is never committed.
- **Secure error handling:** internal exceptions are never returned to the client. Clients get `{"status": "error", "message": "Unable to process your request."}`; full stack traces go to server-side logs only.
- **CORS:** restricted to known origins (`CORS_ORIGINS`), not wildcarded in production.
- **Data minimization:** only the data required for the specific request is collected or processed (ties directly into §9).
- **Logging rule — never log:** passwords, JWT secrets, API keys, access tokens, or unnecessary financial/personal information.
- **Frontend isolation:** React never receives database credentials or direct DB/vector- store access — everything is mediated by FastAPI.

Pre-production security checklist

- ✓ Authentication implemented and enforced on all protected routes
- ✓ Authorization checks scoped to the authenticated user
- ✓ Input validation on every endpoint
- ✓ Rate limiting configured
- ✓ HTTPS enabled end-to-end
- ✓ Secrets in environment/secret storage only, `.env` not committed
- ✓ Sensitive/financial data excluded from logs
- ✓ Generic error messages returned to clients; details logged server-side only
- ✓ CORS restricted to known origins

12. Anti-Hallucination Controls

Core rule: the chatbot answers from **retrieved, approved knowledge** — it does not invent HisabDo features, policies, or procedures.

Rule	Behavior
Do not guess	If confidence/similarity is insufficient, return a safe fallback instead of an invented answer.
Use trusted sources only	Answers are grounded in KB chunks retrieved via RAG — not general LLM knowledge.
Do not invent features	The bot must not claim a feature exists unless it's present in an approved KB document.
Handle uncertainty	Ambiguous queries trigger a clarifying question, not an assumption.
Treat user context carefully	User-supplied context improves relevance but is never treated as a verified fact on its own.
Grounded prompting	The LLM is only ever given retrieved chunks as context for generation — never asked to answer "from memory."

Confidence-based handling (target model for the retrieval step):

High similarity → generate grounded answer

Low similarity → ask clarifying question / safe fallback

Very low / no match → human escalation

Exact thresholds should be tuned from real evaluation data (§14), not fixed arbitrarily.

Example — unsupported query:

User: Can HisabDo do <undocumented feature>?

Bot: I don't have verified information about that feature. Please check the
available help documentation or contact support for confirmation.

Monitoring: fallback rate, escalation rate, and flagged incorrect answers should be reviewed regularly to catch KB gaps and retrieval-quality regressions.

13. Error Handling and Fallback Behavior

Reliable answer available → Answer directly

Request unclear → Ask a clarifying question

No reliable answer available → Safe fallback response (no invented content)

Complex / unresolved issue → Escalate to human support

Level 1 — Clarification

User: "It is not working."

Bot: "Could you tell me which feature is not working, and what error message
you are seeing?"

Level 2 — Safe fallback

"I don't have enough verified information to answer this accurately. Please
provide more details or check the available help information."

Level 3 — Human escalation (triggered when: the issue stays unresolved after clarification, the request is technically complex, it needs account-specific action, the AI can't produce a reliable answer, or the user explicitly asks for a human):

"I'm unable to resolve this issue with the available information. Your request should be reviewed by human support. Please use the available support channel to continue."

Escalation data: only non-sensitive fields are logged for escalation — the support question, relevant error info, the feature involved, conversation context, and a timestamp. No raw financial data or credentials (§9, §11).

API-level error handling: invalid/missing input → 400 with a validation message; auth failures → 401/403; unexpected server errors → generic 500 message to the client, full detail server-side only (§11).

14. Testing Results

Functional / intent test cases

Test Case	Scenario	Input	Expected Result	Status
TC-01	Feature Guidance	"How can I add a new expense?"	Relevant instructions	Pass
TC-02	Step-by-Step Help	"Tell me step by step how to manage my expenses."	Sequential instructions	Pass
TC-03	Error Assistance	"I am getting an error while adding an expense."	Troubleshooting guidance	Pass
TC-04	FAQ	"What is HisabDo?"	Approved information	Pass
TC-05	Context-Aware Help	"I can't find the required option." + context	Contextual response	Pass
TC-06	Personalized Recommendation	{expenses:[...]}	Recommendation generated	Pass
TC-07	Missing Input	empty query	Validation error (400)	Pass
TC-08	Unsupported Query	Off-topic request	Safe fallback, no fabrication	Pass

RAG retrieval evaluation (12-query labeled set)

Metric	Result
Hit@1	83.3%
Hit@3	100%
Mean Reciprocal Rank	0.903

Reproduced twice (once during development, once from a clean virtual environment) with identical results.

Conclusion: current results demonstrate expected behavior across common support scenarios and confirm the retrieval layer surfaces the correct chunk within the top 3 every time in the labeled set. Further testing is still needed with real user queries, malformed/adversarial input, ambiguous phrasing, and multilingual (Urdu / Roman Urdu) queries before production sign-off.

15. Current POC Limitations

- **Embedding substitution:** TF-IDF is used in place of Sentence Transformers due to a sandboxed environment blocking huggingface.co — documented in §7, needs to be swapped before production for better semantic recall.
- **No live KB indexing:** knowledge-base edits require re-running ingestion manually; there's no incremental/live index update yet.
- **No production auth hardening:** rate limiting, HTTPS enforcement, and full authorization-scoping still need to be wired end-to-end before deployment (§10–11).
- **Language coverage:** primarily tuned for English; Urdu, Roman Urdu, and mixed-language queries (e.g. bijli ka bill pay kiya) are expected to perform worse until the KB and embedding model are extended.
- **Static knowledge base:** the system does not learn from new conversations; it only improves when the KB content itself is updated and re-indexed.
- **Escalation is not yet wired to a real ticketing system:** escalation currently produces a message to the user, not an actual support-queue entry.
- **FAISS is local-only:** fine for the prototype's scale; Qdrant migration (§1, planned) is required before multi-instance/production scale.
- **Confidence $\neq$ correctness:** a high similarity score reduces but does not eliminate the risk of an imperfect answer — human review of flagged/low-confidence cases is still recommended during rollout.

16. Developer Setup / Integration Checklist

Prerequisites: Python 3.11, Node.js 20 LTS, Git, MongoDB (local or Atlas), Ollama, Docker Desktop (optional, for docker-compose.yml).

Backend

```
python -m venv venv
```

```
source venv/bin/activate          # Windows: .\venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
```

```
pip install -r backend/requirements.txt

uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# Frontend

cd frontend

npm install

npm run dev
```

Startup order: MongoDB → Ollama → FastAPI → Node/Express (if used) → React.

Environment (copy .env.example → .env, then edit secrets):

```
JWT_SECRET=CHANGE_THIS_IN_LOCAL_ENV

MONGODB_URI=mongodb://localhost:27017/hisabdo

OLLAMA_BASE_URL=http://localhost:11434

OLLAMA_MODEL=llama3.2:3b

EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2

VECTOR_STORE=faiss

CORS_ORIGINS=http://localhost:5173
```

Validation before calling it "integrated":

- /health, /health/database, /health/llm, /health/vector-store all return OK
- Sample RAG query returns a grounded answer with source chunks
- Unsupported/off-topic query returns the safe-fallback response, not a fabricated one
- Missing/empty input returns a 400 validation error, not a 500
- JWT-protected endpoint rejects an unauthenticated request
- No secrets present in git diff before every commit